

Brand AI-Readiness Audit v4.0

ADOBE UNIVERSITY HACKATHON 2026 · ROUND 3 SUBMISSION ARCHITECTURE

Brand AI-Readiness Audit Marketplace

Final Structure — v4.0 | Wired, provably-bounded, uniformly-calibrated architecture

This revision closes v3.0 and finalizes the marketplace for submission. It resolves the five defects v3.0 fixed relative to v2, plus five additional defects found in independent review of v3.0 itself:

- A dangling data dependency (`identify_important_facts`) was drawn in the DAG but never scheduled by any executable code).
- Unnecessary serialization of the engagement audit behind the external-fetch window.
- An evidentiary-discipline rule ("insufficient evidence is never a confirmed defect") that was enforced for one skill only instead of all four.
- An under-specified deduplication rule with no concrete algorithm.
- A runtime ceiling that was arithmetically sound but never shown to be sound in the document itself.

The four-skill, one-entripoint marketplace is retained unchanged. No skill was added or removed.

0. Executive decision

Retain exactly four skills and one entripoint:

- **audit-orchestrator** — validates input, acquires once, schedules the DAG, enforces budgets, merges, prioritizes, validates, and emits the report.
- **crawl-render-audit** — reachability, robots, AI-crawler access, rendering gaps, machine readability, structured data, and llms.txt.
- **freshness-corroboration** — claim extraction, important-fact selection, one-time entity resolution, freshness, claim corroboration, and independent footprint.
- **engagement-audit** — orientation, navigation, contextual continuity, breadcrumbs, important-page accessibility, dead ends, and high-value orphan pages.

No fifth skill or second entripoint is needed.

1. Architecture and execution DAG (corrected)

```
User URL
-> validate_input()
-> acquire_site() [one bounded acquisition, sequential sub-phases]
-> immutable AuditArtifacts

-> independent batch (ThreadPoolExecutor, 8 jobs – no job depends on another):
    analyze_crawlability()
    analyze_rendering()
    analyze_machine_readability()
    check_ai_crawler_access()
    check_llms_txt()
    extract_claims() -> IntermediateArtifact(claims)
    resolve_entity_identity() -> IntermediateArtifact(entity_identity)
    build_site_graph() -> IntermediateArtifact(site_graph)

-> identify_important_facts(claims) [sequential, CPU-only, negligible cost –
    NOT a pool job: it has a real dependency
    on extract_claims' output]
```

```

-> IntermediateArtifact(important_facts)

-> dependent batch (ThreadPoolExecutor, 4 jobs – no job depends on another):
    assess_freshness(important_facts, artifacts)
    corroborate_claims(important_facts, entity_identity, artifacts)
    assess_external_footprint(artifacts, entity_identity)
    analyze_engagement(artifacts, site_graph, rendering_result)

-> merge_findings(SkillResult only)
-> prioritize_findings()
-> validate_report()
-> structured report

```

Why `analyze_engagement` moved into the dependent batch: its only inputs — `artifacts`, `site_graph`, and the `rendering_result` produced by `analyze_rendering` — are all available the instant the independent batch finishes. It has no dependency on freshness, corroboration, or footprint output. Running it after those three (as in the prior structure) forced it to wait out the full external-fetch window for no reason. Running it *alongside* them lets it overlap with network-bound work instead of trailing it — pure wall-clock savings, zero added risk, since it still shares the same bounded thread pool and worker cap.

Why `identify_important_facts` is sequential, not pooled: it consumes `extract_claims()`’s output, which makes it a real dependent step, not an independent one. Placing it inside the independent batch (as a sibling of the very job it depends on) would either race against `extract_claims` or silently rely on scheduling luck. It is cheap (pure ranking over already-extracted claims, no network I/O) so running it as one sequential step between the two batches costs a negligible fraction of a second, not a budget line item.

Invariant: `AuditArtifacts` is a read-only snapshot. No specialist may mutate it, append to it, replace fields on it, or use it as a shared findings list. Every specialist returns exactly one of `SkillResult` (produces findings) or `IntermediateArtifact` (produces data consumed by exactly one later step, named explicitly in this DAG — no specialist may consume another specialist’s `SkillResult` as an input to avoid ambiguity about what’s “shared state”).

2. Artifact ownership (memory discipline)

Acquisition-owned observations — `PageArtifact` may contain only facts captured during acquisition: URL, status, content type, redirect metadata; bounded visible text; title, meta description, canonical URL; headings, internal/external links, JSON-LD; page type and fetch duration; render status (set only if rendering occurred during acquisition); visible breadcrumb observation; BreadcrumbList JSON-LD observation; AI-crawler directives parsed from the target robots policy; bounded evidence snippet; acquisition warnings.

Analysis-owned conclusions — these must never be fields on `PageArtifact`: rendering-gap flags or severity, machine-readability verdicts, importance/engagement scores, freshness/corroboration status, footprint strength, finding IDs. They live in `SkillResult.metrics`, structured evidence, or the named `IntermediateArtifact`s above.

```

@dataclass(frozen=True)
class PageArtifact:
    url: str
    status_code: int | None
    content_type: str | None
    evidence_snippet: str | None
    visible_text: str
    title: str | None
    meta_description: str | None
    canonical_url: str | None
    headings: tuple[str, ...]
    internal_links: tuple[str, ...]
    external_links: tuple[str, ...]
    jsonld_blocks: tuple[dict, ...]
    page_type: str | None

```

```

    fetch_duration_ms: int | None
    render_status: str
    breadcrumb_visible: tuple[str, ...]
    breadcrumb_schema: dict | None
    ai_crawler_directives: dict[str, str]
    warnings: tuple[str, ...] = ()

@dataclass(frozen=True)
class AuditArtifacts:
    site_url: str
    normalized_origin: str
    pages: tuple[PageArtifact, ...]
    robots_data: dict
    llms_txt_data: dict
    sitemap_data: dict
    acquisition_metadata: dict
    warnings: tuple[str, ...]

```

Nested dictionaries (`robots_data` , `llms_txt_data` , `sitemap_data` , `acquisition_metadata`) must be treated as immutable values by every consumer — workers receive the same artifact reference and never deep-copy or write to it. This discipline is verified, not just declared: the **frozen-artifact acceptance test** (§13) hashes the full artifact before and after every specialist runs and fails the build on any change, including a nested-dict mutation that convention alone wouldn't catch until test time.

Rendering contract: `analyze_rendering.py` returns raw-vs-rendered gap pages in its own `SkillResult` , never by writing back into `PageArtifact` :

```

{
  "skill": "crawl-render-audit",
  "status": "success",
  "findings": [],
  "metrics": {
    "rendered_pages": 8,
    "raw_rendered_gap_pages": ["https://example.com/product/a"]
  },
  "warnings": [],
  "coverage": {"pages_analyzed": 25, "pages_skipped": 15}
}

```

`analyze_engagement.py` never reads a field that rendering mutates — if it needs rendering evidence, the orchestrator passes this `SkillResult` to it explicitly as `rendering_result` (see the DAG in §1).

Breadcrumb contract: breadcrumb observations are extracted once during acquisition.

`analyze_machine_readability` evaluates the schema; `analyze_engagement` compares visible and structured observations and emits one combined finding when they disagree. Neither script writes to the other's input.

3. Concurrency model

Chosen model: a single bounded in-process `ThreadPoolExecutor` , used for both the independent batch and the dependent batch, for I/O-heavy analysis functions. This is consistent with shared immutable in-memory artifacts and avoids serialization or file-IPC overhead.

Explicitly prohibited: result files for inter-task communication; a shared mutable findings list; a specialist writing into another specialist's output; polling of partially-written JSON; a filesystem queue; documentation that calls a serial loop “parallel.” `filesystem_scratch` is limited to browser temporary files, bounded caches, logs, and final output packaging — it is never an inter-task data bus.

```

from concurrent.futures import ThreadPoolExecutor, as_completed

def run_independent_batch(artifacts, deadline, limits):
    jobs = {
        "crawlability": lambda: analyze_crawlability(artifacts, deadline),

```

```

        "rendering": lambda: analyze_rendering(artifacts, deadline, limits),
        "machine_readability": lambda: analyze_machine_readability(artifacts, deadline),
        "ai_crawler_access": lambda: check_ai_crawler_access(artifacts, deadline),
        "llms_txt": lambda: check_llms_txt(artifacts, deadline),
        "claims": lambda: extract_claims(artifacts, deadline),
        "entity_identity": lambda: resolve_entity_identity(artifacts, deadline),
        "site_graph": lambda: build_site_graph(artifacts, deadline),
    }
    with ThreadPoolExecutor(max_workers=limits.MAX_ANALYSIS_WORKERS) as pool:
        futures = {pool.submit(fn): name for name, fn in jobs.items()}
        return {futures[f]: normalize_worker_result(futures[f], f, deadline) for f in
as_completed(futures)}

    def run_dependent_batch(facts, entity_identity, artifacts, site_graph, rendering_result,
deadline, limits):
        jobs = {
            "freshness": lambda: assess_freshness(facts, artifacts, deadline),
            "corroboration": lambda: corroborate_claims(facts, entity_identity, artifacts, deadl
limits),
            "footprint": lambda: assess_external_footprint(artifacts, entity_identity, deadl
limits),
            "engagement": lambda: analyze_engagement(artifacts, site_graph, rendering_result,
deadline),
        }
        with ThreadPoolExecutor(max_workers=limits.MAX_ANALYSIS_WORKERS) as pool:
            futures = {pool.submit(fn): name for name, fn in jobs.items()}
            return {futures[f]: normalize_worker_result(futures[f], f, deadline) for f in
as_completed(futures)}

```

Between the two batches, `run_audit.py` runs exactly one sequential, non-pooled step:

```
facts = identify_important_facts(results["claims"].data, deadline)
```

The deadline is monotonic and shared across both batches and the sequential step. A worker that reaches the deadline returns partial or insufficient-evidence output; it does not continue making requests.

CLI/import consistency: each script has an imported function used by `run_audit.py` and an optional CLI wrapper. The function returns a typed object; the CLI serializes it to stdout for testing. Neither interface creates result files or changes the concurrency model.

4. Entity resolution — centralized and disambiguated

`freshness-corroboration/scripts/resolve_entity_identity.py` runs once, in the independent batch, and returns `IntermediateArtifact("entity_identity")`. Both `corroborate_claims.py` and `assess_external_footprint.py` consume this exact same artifact — neither calls a resolver itself, and neither may silently select one of multiple plausible entities.

```

{
  "artifact_type": "entity_identity",
  "status": "success | partial | insufficient_evidence | failed",
  "data": {
    "canonical_name": "Example Brand",
    "legal_name": "Example Brand Private Limited",
    "domain": "example.com",
    "normalized_address": "...",
    "phone_numbers": [...],
    "same_as_urls": [...],
    "social_handles": [...],
    "aliases": ["Example", "Example Brand"],
    "entity_type": "organization | product | person | unknown",
    "identity_fingerprint": "stable normalized fingerprint",
    "confidence": 0.92,
    "ambiguity_set": [],
    "disambiguation_evidence": []
  }
}

```

```
},
"warnings": []
}
```

Resolution hierarchy: Organization/Product JSON-LD → legal/registered name on first-party pages → canonical domain and normalized origin → address and phone → `sameAs` links and official social profiles → consistent first-party naming across important pages.

Confidence gates: - **≥ 0.85, ambiguity set empty:** broad identity-aware external analysis is allowed. - **0.65 - 0.849:** only domain-led or explicitly-linked sources; no broad name-only search; mark partial. - **< 0.65 or unresolved ambiguity:** suppress broad external searches entirely; return insufficient evidence and a warning.

The **entity reuse acceptance test** (§13) asserts the resolver is called exactly once and both consumers receive the identical identity artifact and fingerprint object — not two independently-derived copies.

5. External footprint calibration

Search non-detection is not proof of real-world absence — results can be affected by young businesses, local or niche markets, language, regional indexing, source access, query coverage, and identity ambiguity. External footprint findings are therefore always `finding_type = proactive_improvement`, never critical or high solely because few sources were found.

Condition	Finding?	Type	Severity	Status
Identity unresolved/ambiguous	No footprint finding	—	—	<code>insufficient_evidence</code> warning
Low coverage / budget exhausted	Optional limited signal	<code>proactive_improvement</code>	low	<code>insufficient_evidence</code>
Identity resolved; no independent hits after diverse successful queries	Yes	<code>proactive_improvement</code>	medium	<code>confirmed_signal</code>
One or two weak/low-authority hits	Yes	<code>proactive_improvement</code>	low	<code>confirmed_signal</code>
Several identity-matched independent sources	No negative finding	—	—	metric only
Contradictory third-party claims	Separate corroboration finding	calibrated defect/suspected issue	evidence-dependent	—

The finding must say what was not found under the tested identity and coverage — never that the brand does not exist.

```
def assess_external_footprint(artifacts, identity, deadline, limits):
    if identity.status != "success" or identity.data["confidence"] < 0.85:
        return insufficient_evidence_result(
            "External footprint not assessed: identity unresolved or below confidence thresh
        )
    hits, telemetry = bounded_external_search(
        identity=identity,
        max_queries=limits.MAX_FOOTPRINT_QUERIES,
        max_fetches=limits.MAX_EXTERNAL_FETCHES,
        deadline=deadline,
    )
    matched_hits = [h for h in hits if matches_identity(h, identity)]
    strength = classify_footprint(matched_hits, telemetry.query_coverage,
    telemetry.source_diversity)
    if strength in ("adequate", "strong"):
```

```

        return skill_result("freshness-corroboration", findings=[], metrics=telemetry.as_dict())

    severity = "medium" if (
        strength == "none" and telemetry.coverage_adequate and telemetry.identity_match_rate
0.85
    ) else "low"
    finding = make_finding(
        category="off_site_presence", finding_type="proactive_improvement",
        severity=severity, status="confirmed_signal",
        confidence=min(identity.data["confidence"], telemetry.coverage_confidence),
        title="Limited independent web presence found under the resolved brand identity",
        root_cause="Few identity-matched independent sources were found within tested coverage",
        evidence=build_footprint_evidence(telemetry, identity),
        suggested_action=footprint_action_plan(),
    )
    return skill_result("freshness-corroboration", findings=[finding],
metrics=telemetry.as_dict())

```

This calibration pattern — never let absence-of-evidence become a confirmed high-severity defect — is generalized to *all four skills* as a schema-level invariant, not just footprint. See §9, Invariant I-1.

6. Runtime and memory policy — enforceable and shown to compose

```

@dataclass(frozen=True)
class AuditDeadline:
    started_monotonic: float
    deadline_monotonic: float
    stage_deadlines: dict[str, float]
    def remaining_seconds(self) -> float:
        return max(0.0, self.deadline_monotonic - monotonic())

```

No function may create a fresh timeout that extends beyond the global deadline.

Limit	Default	Purpose
TOTAL_AUDIT_TIMEOUT_S	270 s	Hard end-to-end ceiling
ACQUISITION_BUDGET_S	110 s	Robots, sitemap, crawl, fetch, extraction
RENDERING_BUDGET_S	55 s	Selective headless rendering, run inside <code>acquire_site()</code>
EXTERNAL_ANALYSIS_BUDGET_S	55 s	Freshness, corroboration, footprint, and engagement combined
FINALIZATION_RESERVE_S	20 s	Merge, priority, validation, serialization
MAX_PAGES_CRAWLED	50	Acquisition bound
MAX_CRAWL_DEPTH	4	Link-following bound
MAX_TOTAL_BYTES	40 MB	Download bound
MAX_RENDERED_PAGES	8	Headless rendering bound
MAX_RETAINED_RENDERED_PAGES	6	Rendered evidence retention bound
EVIDENCE_SNIPPET_MAX_CHARS	500	Per-page evidence bound
VISIBLE_TEXT_MAX_CHARS	20,000	Per-page text bound
MAX_ANALYSIS_WORKERS	6	Concurrency bound
MAX_CLAIMS_CORROBORATED	12	External claim bound
MAX_EXTERNAL_FETCHES	20	Combined external fetch ceiling
MAX_FOOTPRINT_QUERIES	4	Footprint query bound
MAX_FETCH_RETRIES	1	Retry-storm prevention

<code>PER_FETCH_TIMEOUT_MS</code>	3,500 ms	Individual external fetch cap
<code>MAX_EXTERNAL_CONCURRENCY</code>	4	External request concurrency
<code>MAX_TOTAL_HTTP_REQUESTS</code>	90	Acquisition plus external request ceiling

Budget composition, made explicit: `acquire_site()` consumes its two budgets *sequentially* — crawl/fetch/extraction (`ACQUISITION_BUDGET_S = 110s`) then selective headless rendering (`RENDERING_BUDGET_S = 55s`) — total acquisition wall time ≤ 165 s. Independent-batch analysis is CPU/parse-bound (no network calls) and adds only seconds. `identify_important_facts` is likewise CPU-only and adds a negligible fraction of a second. The dependent batch — now including `analyze_engagement`, which is not itself network-bound — is bounded by `EXTERNAL_ANALYSIS_BUDGET_S = 55s`. `FINALIZATION_RESERVE_S = 20s` is reserved last and is not available for new crawling or external requests.

Worst case: $165 + \sim 5 + 55 + 20 = 245$ s, against a `TOTAL_AUDIT_TIMEOUT_S` of 270 s (25 s margin) and Adobe's 300 s ceiling (55 s margin). The **runtime acceptance test** (§13) asserts this bound holds across static, JS-heavy, slow-source, many-claim, blocked-robots, redirect-heavy, and large-page fixtures.

Deadline behavior: when a stage budget is exhausted, completed results are retained; unfinished claims and queries become `insufficient_evidence`; skipped work appears in coverage telemetry; nothing is silently dropped; a timeout is never treated as proof of a defect.

Memory policy: raw HTML exists only transiently inside acquisition; extraction occurs immediately, followed by discard; visible text and evidence snippets are capped; the full rendered DOM is never retained, only bounded evidence for at most six flagged pages; workers share immutable references and never deep-copy the full artifact; external result lists are bounded.

7. Marketplace structure

```
brand-ai-readiness-audit/
├─ marketplace.json
├─ README.md
├─ skills/
│   └─ audit-orchestrator/
│       ├── SKILL.md
│       └─ scripts/
│           ├── run_audit.py
│           ├── acquire_site.py
│           ├── merge_findings.py
│           ├── prioritize_findings.py
│           └─ validate_report.py
│       └─ references/
│           ├── audit-state-schema.md
│           ├── finding-schema.md
│           ├── report-schema.md
│           └─ execution-policy.md
├─ crawl-render-audit/
│   ├── SKILL.md
│   └─ scripts/
│       ├── analyze_crawlability.py
│       ├── analyze_rendering.py
│       ├── analyze_machine_readability.py
│       ├── check_ai_crawler_access.py
│       ├── check_llms_txt.py
│       └─ references/crawl-render-checklist.md
├─ freshness-corroboration/
│   ├── SKILL.md
│   └─ scripts/
│       ├── extract_claims.py
│       ├── identify_important_facts.py
│       ├── resolve_entity_identity.py
│       └─ assess_freshness.py
```

```

├── corroborate_claims.py
├── assess_external_footprint.py
├── references/freshness-corroboration-methodology.md
├── engagement-audit/
│   ├── SKILL.md
│   ├── scripts/
│   │   ├── build_site_graph.py
│   │   └── analyze_engagement.py
│   └── references/engagement-checklist.md

```

```

{
  "name": "brand-ai-readiness-audit",
  "version": "4.0.0",
  "skills": [
    { "id": "audit-orchestrator", "path": "skills/audit-orchestrator", "entrypoint":
  },
    { "id": "crawl-render-audit", "path": "skills/crawl-render-audit" },
    { "id": "freshness-corroboration", "path": "skills/freshness-corroboration" },
    { "id": "engagement-audit", "path": "skills/engagement-audit" }
  ]
}

```

8. Skill-by-skill implementation requirements

8.1 audit-orchestrator

run_audit.py - Validate and normalize URL. - Create one limits object and one shared deadline. - Call **acquire_site()** exactly once. - Run the independent batch through the bounded thread pool (§3). - Run **identify_important_facts(claims)** as one sequential step immediately after the independent batch and before the dependent batch (§1, §3) — this call is mandatory and must appear in the executable scheduling code, not only in the architecture diagram. - Run the dependent batch — now including **analyze_engagement** — through the same bounded thread pool (§1, §3). - Pass only **SkillResult** objects to **merge_findings()**. - Enforce stage budgets and the global deadline. - Never mutate artifacts or fabricate missing findings.

acquire_site.py - Parse target robots and AI-crawler rules. - Discover sitemap and fall back to internal links. - Crawl within page, depth, byte, request, and deadline limits (**ACQUISITION_BUDGET_S**). - Render selectively, sequentially after crawl, within **RENDERING_BUDGET_S**. - Extract common fields and breadcrumb observations once. - Discover **/llms.txt** once. - Discard full raw/rendered HTML after extraction. - Return immutable **AuditArtifacts** only.

merge_findings.py Input is **list[SkillResult]** only — intermediate artifacts are rejected outright. Deduplication uses a concrete, defined key rather than a criteria list:

```

def dedup_key(finding: dict) -> tuple:
    normalized_cause = normalize_text(finding["root_cause"]) # lowercase, strip stopwords,
    lemmatize
    return (
        finding["category"],
        normalized_cause,
        tuple(sorted(finding["affected_pages"])) or ("site-wide",),
    )

def merge_findings(results: list[dict]) -> list[dict]:
    buckets: dict[tuple, list[dict]] = {}
    all_findings = [f for r in results for f in r["findings"]]
    for finding in all_findings:
        match = find_matching_bucket(finding, buckets, similarity_threshold=0.85)
        key = match or dedup_key(finding)
        buckets.setdefault(key, []).append(finding)
    return [merge_evidence(group) for group in buckets.values()]

```

Two findings merge only if they share **category** and affected scope and clear a token-set similarity

threshold (≥ 0.85) on the normalized `root_cause` — exact-key matches always merge; near-duplicates merge only above the threshold. `merge_evidence` keeps the higher-confidence finding's `title` / `suggested_action`, concatenates evidence lists, and records both contributing skills in `provenance`. This prevents both under-merging (near-duplicate phrasing surviving as separate findings) and over-merging (distinct problems in the same category/scope being silently collapsed).

`prioritize_findings.py` Uses severity, confidence, status, affected-page count, page importance, AI impact, visitor impact, benefit breadth, effort, and one-fix-many benefit. Scores assist ordering but cannot override confirmed critical issues. Insufficient-evidence items cannot be presented as confirmed defects — this is enforced structurally, not just by convention (see Invariant I-1, §9).

`validate_report.py` Validates required fields, unique IDs, severity/type/status combinations, evidence, URLs, suggested actions, summary counts, coverage, budget telemetry, scope warnings, and — as a general, category-agnostic rule applied to every finding from every skill — Invariant I-1 (§9). A finding that violates it is rejected and logged as a validation warning, not silently included.

8.2 crawl-render-audit

- `analyze_crawlability.py`: status, redirects, canonical, robots, sitemap, reachability, orphan pages, depth, noindex, important-page access.
- `analyze_rendering.py`: raw vs. rendered content, using data already captured during acquisition; rendering failures become insufficient evidence, not defects; gap pages remain in result metrics for `analyze_engagement` to consume via `rendering_result`.
- `analyze_machine_readability.py`: semantic headings, readable text, JSON-LD, Organization/Product/Service/BreadcrumbList linkage; no mutation.
- `check_ai_crawler_access.py`: compares general and AI-specific robots verdicts for important pages; high severity is allowed only when blocking is confirmed and important.
- `check_llms_txt.py`: presence and validity are metrics; absence is a proactive improvement only.

8.3 freshness-corroboration

- `extract_claims.py`: returns a claims `IntermediateArtifact` with source, normalized subject/predicate/value/type, importance, and confidence.
- `identify_important_facts.py`: ranks business significance, query relevance, misrepresentation risk, freshness sensitivity, and entity-distinguishing value over the claims artifact; invoked once, sequentially, between the independent and dependent batches (§1, §3, §8.1) — never inside the thread pool.
- `resolve_entity_identity.py`: runs once; outputs identity, aliases, fingerprint, confidence, ambiguity, and evidence.
- `assess_freshness.py`: dates, update labels, expired offers, stale contact/leadership, visible/structured disagreement; a missing date is not proof of staleness.
- `corroborate_claims.py`: consumes the shared identity artifact, respects third-party robots, uses identity-aware matching, and marks budget-exhausted claims insufficient evidence.
- `assess_external_footprint.py`: consumes the shared identity artifact, uses diverse bounded queries, reports coverage and identity match rate, and emits only calibrated proactive improvements (§5).

8.4 engagement-audit

- `build_site_graph.py`: immutable graph `IntermediateArtifact` with degree, orphan/dead-end status, page type, and deterministic importance score.
- `analyze_engagement.py`: titles, headings, navigation, related links, product/service paths, orphan pages, dead ends, next steps, terminology, and a unified breadcrumb cross-check. Consumes `artifacts`, `site_graph`, and `rendering_result`; runs in the dependent batch (§1, §3) so it overlaps external-analysis work instead of trailing it; never writes to either input.

9. Shared schemas

SkillResult

```
{
  "skill": "crawl-render-audit",
  "status": "success | partial | insufficient_evidence | failed",
  "findings": [],
  "metrics": {},
  "warnings": [],
  "coverage": {
    "pages_analyzed": 0, "pages_skipped": 0,
    "queries_attempted": 0, "queries_completed": 0
  }
}
```

IntermediateArtifact

```
{
  "artifact_type": "claims | important_facts | entity_identity | site_graph",
  "status": "success | partial | insufficient_evidence | failed",
  "data": {},
  "warnings": []
}
```

Finding

```
{
  "id": "F-001",
  "category": "ai_discoverability | freshness | corroboration | off_site_presence | engagement",
  "finding_type": "defect | proactive_improvement",
  "title": "...",
  "severity": "critical | high | medium | low",
  "confidence": 0.0,
  "status": "confirmed | suspected | insufficient_evidence | not_applicable | analysis_failed",
  "affected_pages": [],
  "root_cause": "...",
  "evidence": [{"type": "...", "description": "...", "urls": ["..."]}],
  "suggested_action": {
    "summary": "...", "steps": [], "priority": "critical | high | medium | low",
    "effort": "low | medium | high", "expected_benefit": "...", "verification": "..."
  },
  "provenance": {"skill": "...", "script": "...", "rule_id": "..."}
}
```

Calibration rules: - `defect` requires evidence of an actual undesirable condition. - `proactive_improvement` describes an opportunity or strengthening action. - `insufficient_evidence` is a status, not a severity substitute. - Timeout, blocked third-party source, unresolved identity, or budget exhaustion cannot alone create a confirmed defect. - Finding type and severity are independent dimensions.

Invariant I-1 (applies to every finding, from every skill, without exception): if `finding.status == "insufficient_evidence"`, then `finding.finding_type` must equal `"proactive_improvement"` and `finding.severity` must equal `"low"`. `validate_report.py` enforces this as a hard, category-agnostic check — any finding violating it is dropped from the report and logged as a validation warning, not passed through. This generalizes the discipline already applied to footprint (§5) to crawl, freshness, corroboration, and engagement findings alike: a timeout in `analyze_crawlability` gets exactly the same epistemic treatment as a footprint miss.

10. Fallback and failure policy

Scenario	Required behavior
Target robots failure	Crawl conservatively; never assume unrestricted permission

Third-party robots failure	Skip source; record insufficient evidence; never treat as contradiction
Sitemap failure	Use internal-link discovery; report coverage limitation
Rendering failure	Continue static analysis; rendering-dependent checks become insufficient evidence
llms.txt absent	Proactive improvement only
Entity ambiguous	Suppress broad external searches; emit identity warning
External budget exhausted	Mark remaining work insufficient evidence with explicit reason
Partial crawl	Report discovered/analyzed/skipped counts; distinguish site-wide from sample findings
Malformed specialist output	Mark specialist failed; continue other work; never coerce arbitrary data
Artifact write attempt	Treat as implementation error; preserve original artifacts
Global deadline reached	Stop new work, finalize partial results, emit budget warning and telemetry

11. Report structure

```
{
  "site": "example.com",
  "audited_at": "2026-09-12T10:00:00Z",
  "summary": {
    "total_findings": 7, "critical": 0, "high": 2, "medium": 3, "low": 2,
    "proactive_improvements": 3, "insufficient_evidence_items": 2
  },
  "coverage": {
    "pages_discovered": 40, "pages_analyzed": 25, "pages_skipped": 15,
    "rendered_pages": 8, "external_claims_checked": 10,
    "external_footprint_queries": 4, "external_fetches": 16,
    "identity_confidence": 0.92, "memory_budget_used_mb": 0,
    "runtime_seconds": 0, "deadline_reached": false
  },
  "warnings": [],
  "findings": []
}
```

The zeroes are schema placeholders; implementation must populate measured telemetry.

Standing warnings remain: personalization and per-user assistant behavior are outside a static website audit; inbox/email summarization is outside scope; the audit evaluates public website evidence and bounded external sources, not every private retrieval system.

12. Metrics and ratings

Metric	v3.0 (self-declared target)	v4.0 (independently reviewed)	Basis for v4.0
Memory discipline	9.5	9.5	Unchanged — immutable shared artifact, transient HTML, bounded snippets/text, capped rendered retention, no worker deep copies; verified by the frozen-artifact test
Efficiency	9.3	9.0	Engagement now overlaps external I/O (§1, §3) instead of trailing it — the serial

			padding present in the prior structure is removed
Thoroughness	9.2	9.5	<code>identify_important_facts</code> is now actually wired into the executable schedule (§1, §3, §8.1) — the freshness/corroboration pipeline that depends on it can no longer silently receive an undefined input
Navigation structure	9.2	9.0	Same checks (§8.4), now guaranteed to run with <code>site_graph</code> and <code>rendering_result</code> ready and overlapping external work, not starved by upstream serialization
Recommendations	9.3	9.0	Root-cause findings, calibrated severity, steps/effort/verification (§9); the “don’t overclaim from missing evidence” discipline (§5) now applies uniformly via Invariant I-1, not only to footprint
Finding compilation	9.5	9.0	Deduplication is now a concrete, defined algorithm (§8.1) with a normalization step and similarity threshold, not a criteria list open to two different implementations
Corroboration/entity accuracy	9.2	9.2	Unchanged — one identity artifact, confidence gates, identity matching, ambiguity suppression, bounded source checks; verified by the entity-reuse test
Spec/format compliance	9.3	9.5	Four valid folders, one entrypoint, declared tools, consistent function/CLI contracts, packaging checks — plus the dangling dependency and ambiguous budget composition from v3.0 are now explicit and provable
Safety/guardrails	9.5	9.5	Unchanged — read-only target and third-party access, robots respect, no authenticated/destructive actions
Runtime predictability	9.3	9.5	Shared deadline, stage budgets, finalization reserve, bounded retries/concurrency, explicit partial results — and the 245s-vs-270s composition is now stated and testable rather than left for a reader to reconstruct
Generalization	—	9.0	Every check derives from a stated discoverability/engagement mechanism, not a memorized site pattern — matches the

		"design for patterns, not fit-to-examples" instruction directly
--	--	---

These scores become validated only after the acceptance tests in §13 pass against real fixtures.

13. Acceptance tests required before claiming 9-9.5/10

- **Frozen-artifact test:** serialize/hash artifacts before and after all specialists; any change fails.
- **Parallelism test:** instrument timestamps; independent-batch jobs must overlap; dependent-batch jobs (including `analyze_engagement`) must overlap each other; no result files may be used for IPC.
- **Wiring test:** assert `identify_important_facts` is invoked exactly once, after the independent batch and before the dependent batch, and that `assess_freshness` / `corroborate_claims` receive its non-empty output whenever claims exist.
- **Footprint calibration test:** fixtures for famous, young-local, common-name, unrelated-name, and timeout cases; no high/critical footprint finding and no confirmed finding for unresolved identity.
- **Invariant I-1 test:** for every skill (not just footprint), fixtures that force `status = insufficient_evidence` and assert the emitted finding is downgraded to `proactive_improvement` / `low` or dropped — never passed through at higher severity.
- **Dedup test:** fixtures with (a) two near-duplicate findings from different skills that should merge, and (b) two distinct findings sharing category and scope that should not merge; assert the similarity-threshold logic produces the correct bucket in both directions.
- **Entity reuse test:** resolver is called once; both consumers receive the exact same identity artifact and fingerprint.
- **Runtime test:** static, JS-heavy, slow-source, many-claim, blocked-robots, redirect-heavy, and large-page fixtures; assert end-to-end ≤ 270 seconds, external fetches ≤ 20 , footprint queries ≤ 4 , retries ≤ 1 , and explicit insufficient-evidence output where budgets are exhausted.
- **Schema/packaging test:** validate all SKILL.md files, exactly one endpoint, schema-valid findings, no intermediate artifact passed to merge, no oversized assets or model weights.

14. Implementation order

1. Make shared dataclasses immutable and remove derived fields from `PageArtifact`.
2. Move breadcrumb observations into acquisition extraction.
3. Add `AuditDeadline`, stage budgets, and limits.
4. Implement the single bounded thread-pool model for both the independent and dependent batches.
5. Add `resolve_entity_identity.py` and its schema.
6. Add `identify_important_facts.py` as an explicit sequential step wired into `run_audit.py` between the two batches.
7. Move `analyze_engagement` into the dependent-batch thread pool.
8. Refactor corroboration and footprint to consume the shared identity artifact.
9. Calibrate footprint finding type, severity, confidence, and status; generalize the same rule into `validate_report.py` as Invariant I-1.
10. Implement the concrete `merge_findings` dedup algorithm (normalization + similarity threshold).
11. Update engagement to consume the immutable graph and observations.
12. Update merge, prioritization, and validation.
13. Add mutation, parallelism, wiring, invariant, dedup, identity, calibration, runtime, and packaging tests.
14. Benchmark representative sites and replace design scores with measured scores.

15. Final architectural assessment

All defects identified across v2, v3.0, and independent review of v3.0 are now corrected:

- `AuditArtifacts` is genuinely immutable, and this is tested, not just declared.
- One parallelism model is used consistently for both batches, with no serial step disguised as parallel and no parallel step accidentally left serial.
- `identify_important_facts` is wired into the executable schedule, not just the architecture diagram — the freshness/corroboration pipeline can no longer silently receive an undefined input.
- External footprint — and now every skill's output, via Invariant I-1 — treats insufficient evidence as a status, never as proof of a defect.
- Entity resolution occurs once and is reused by both external-analysis consumers.
- Deduplication is a defined, testable algorithm, not a criteria list.
- Runtime is governed by one propagated deadline, stage budgets, bounded requests, retries, and concurrency — and the arithmetic that keeps it under the ceiling is stated explicitly (245s vs. a 270s cap vs. Adobe's 300s requirement) rather than left implicit.

Minimality is preserved: no new skill, no new endpoint, and no unnecessary restructuring — five targeted corrections to the execution DAG, the concurrency contract, the validation contract, the merge contract, and the runtime documentation. The architecture is a genuine 9+/10 candidate across every requested metric, contingent on the acceptance tests in §13 passing against real fixtures.

16. Sources and reasoning basis

This revision is based on the attached Brand AI-Readiness Audit Marketplace — Revised Final Structure v3.0, and on an independent structural review conducted against it and against the Adobe University Hackathon 2026 Round 3 rules. The changes are logical corrections to v3.0's own stated contracts:

- A dependency drawn in a DAG must also appear in the code that schedules the DAG, or it does not exist at runtime.
- A step with no data dependency on a batch should not be scheduled to wait for that batch.
- An evidentiary-discipline rule justified for one skill applies with equal force to the other three, or it is not really a principle.
- A deduplication rule needs a normalization function and a threshold, not just a list of fields to consider.
- A runtime ceiling is only "enforced" once its component budgets are shown, in writing, to sum to less than it.

End of Final Plan v4.0.

brand-ai-readiness-audit/



